

Fundamentals behind building a Large Language Model

Tools → Numpy | Pandas | Seaborn | Matplotlib
Programming Lang → Python
Soft → Colab

Core Concept

Scenario → Bank → ₹ 20 Lacs

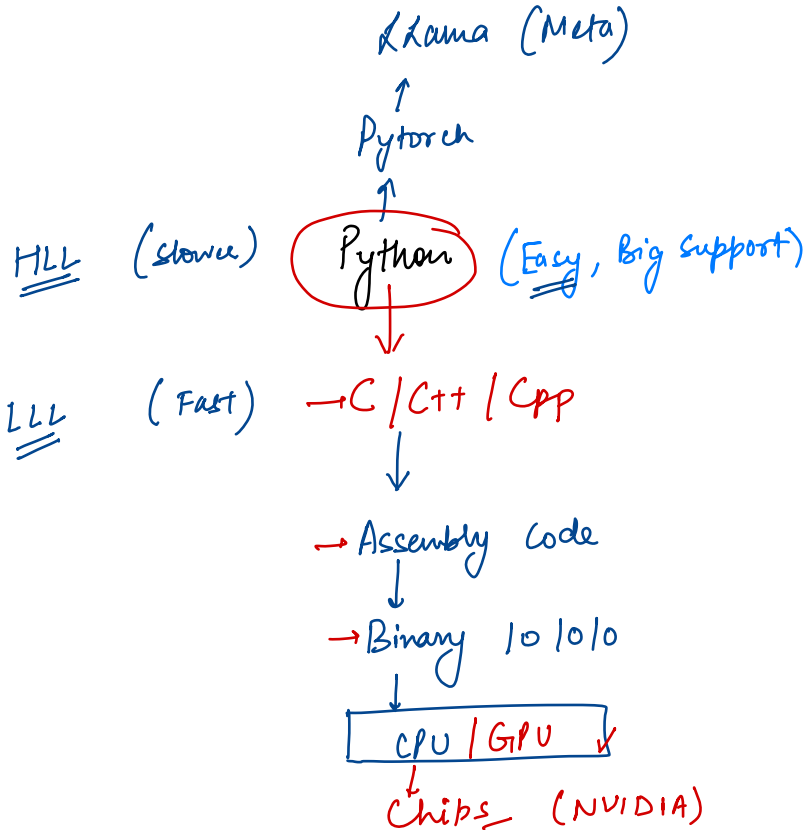
Suraj ← C₁ → 20 Lacs → Delhi, IT → 5 Lac / Month
vijay ← C₂ → 20 Lacs → 7 Lacs / Month
Mallya ↓
Founder of an Airline

History

Data

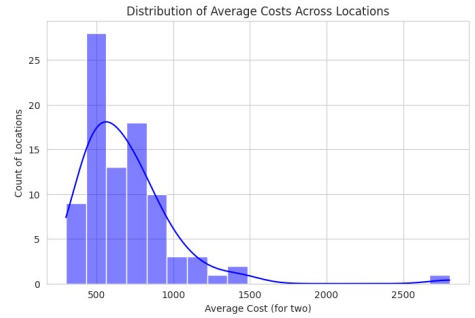
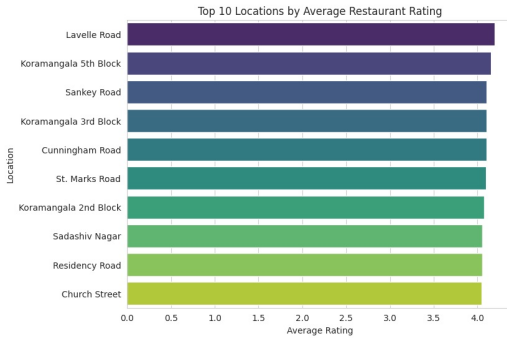
Credit score, Address, Job title, Pin code,
Loans, Loan amt, Prev loans,
Default history, Collateral
(200)

System → First level → give → Human in loop → ✓
History





Reporting



Food delivery & data exploration

- good restaurants
- good ratings
- good offers → Food Payment option
- ETA

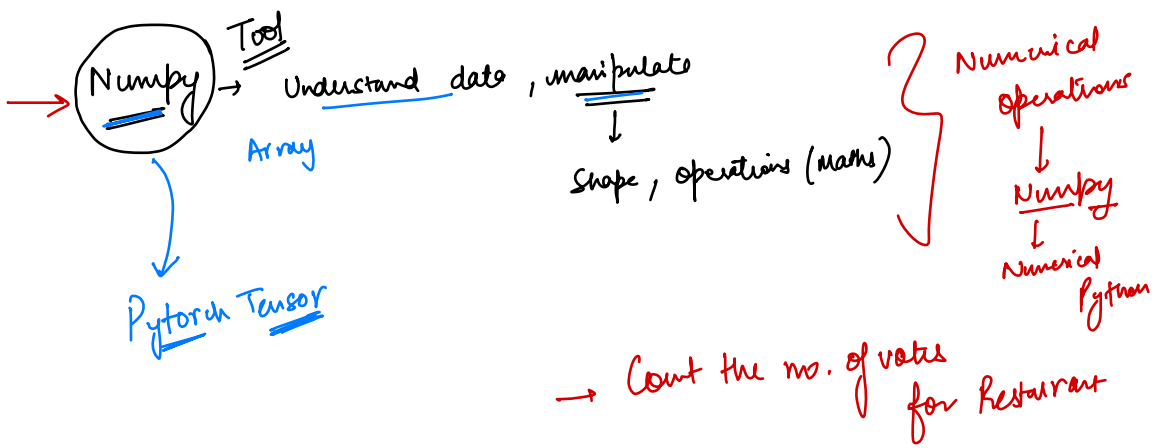
Predict
↓

Name	Location	order	Delivery
—	—	—	—

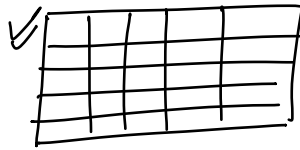
ML → Mistakes
↳ Right decisions



- Missing values
- Age → 12~~x~~ (outlier treatment)
- 1 Bn B → standardize



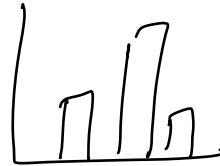
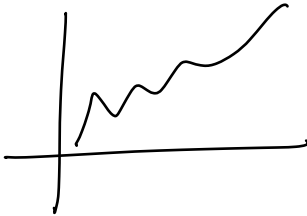
→ CSV, excel, SQL



②
Pandas

Seaborn / matplotlib

③



Zomato

Restaurant



✓ Votes

✓ Cost for 2

How do I create things from Scratch?

Food delivery Day - 1.ipynb

```
x = 3
```

```
y = 5
```

```
z = x + y
```

```
print(z)
```

```
8
```

```
# datatype
```

```
type(x)
```

```
int
```

```
x = 5.1
```

```
type(x)
```

```
float
```

```
y = "Five"
```

```
type(y)
```

```
str
```

```
x + y
```

Type Error - Can only concatenate str (not "float") to str

```
x = '5.1'
```

```
x+y
```

```
'5.1 Five'
```

```
x = 3
```

```
y = 5.1
```

```
z = 3 + 5.1
```

```
z
```

```
8.1
```

```
friends = [2, 3, 4, 5, 6]
```

```
type(friends)
```

```
list
```

```
friends = [2, 'Tina', 'Suroj', True, 6.1, 7]
```

list can accomodate different data types

```
[1, 2, 3] * 2
```

```
[1, 2, 3, 1, 2, 3]
```

```
x = [1, 2, 3]
```

```
for i in x:
```

```
    print(i * 2)
```

```
2
```

```
4
```

```
6
```

numerical operations in python
could be challenging: so solve
this we have numpy

```
import numpy as np
```

```
x = np.array([1, 2, 3])
```

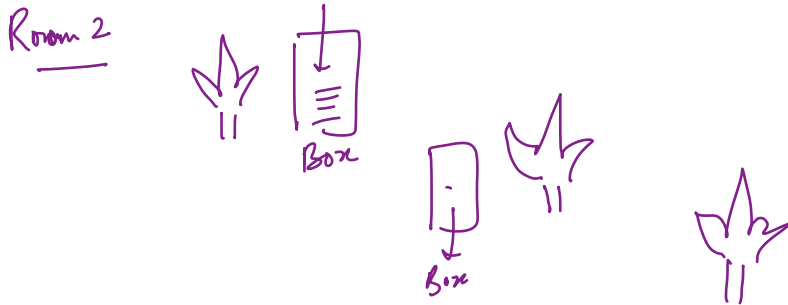
```
type(x)
```

```
numpy.ndarray
```

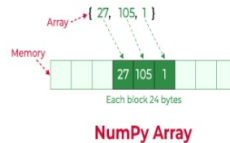
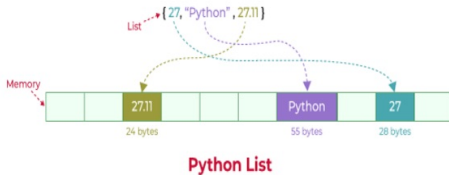
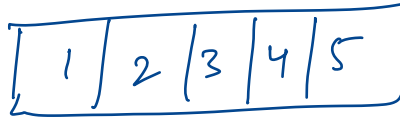
```
x * 2
```

```
array([ 2, 4, 6])
```

Why Numpy is faster?



Numpy
 $x = \text{np.array}([1, 2, 3, 4, 5])$



Can you give different data types together in
 np.array
Elements have homogeneous data type

```
friends = np.array([2, 'Tina', 'Sura', True, 6.1, 7])
```

friends

```
array(['2', 'Tina', 'Sura', 'True', '6.1', '7'], dtype='<U32')
```

#So now we can say in python we say list since it is heterogeneous and in numpy we say array since it is homogeneous because by definition an array should be homogeneous only

```
x = np.array([1, 2, 3, 4, 5.1, 7, 8, 9])
```

x

```
array([1., 2., 3., 4., 5.1, 7., 8., 9.])
```

x * 2

```
array([2., 4., 6., 8., 10.2, 14., 16., 18.])
```

Comparing NumPy Arrays and Python Lists Performance

NumPy arrays are significantly faster for numerical operations compared to standard Python lists, primarily due to their underlying C implementation and optimized memory usage. This example will illustrate this performance difference by performing an element-wise addition on large datasets using both data structures and measuring the execution time.

```
import numpy as np
import time

# Define the size of the arrays/lists for comparison
SIZE = 10_000_000

# --- NumPy Array Performance ---
print(f"Performing operations on {SIZE} elements with NumPy arrays...")

# Create two NumPy arrays
numpy_array_1 = np.arange(SIZE)
numpy_array_2 = np.arange(SIZE)

# Measure time for element-wise addition
start_time_np = time.time()
result_numpy = numpy_array_1 + numpy_array_2
end_time_np = time.time()

numpy_time = end_time_np - start_time_np
print(f"NumPy Array Addition Time: {numpy_time:.6f} seconds")

# Optional: Display a small part of the result to verify
# display(result_numpy[:5])

# --- Python List Performance ---
print(f"\nPerforming operations on {SIZE} elements with Python lists...")

# Create two Python lists
python_list_1 = list(range(SIZE))
python_list_2 = list(range(SIZE))

# Measure time for element-wise addition using a loop
start_time_py = time.time()
result_python = [python_list_1[i] + python_list_2[i] for i in range(SIZE)]
end_time_py = time.time()

python_time = end_time_py - start_time_py
print(f"Python List Addition Time: {python_time:.6f} seconds")

# Optional: Display a small part of the result to verify
# display(result_python[:5])

print(python_time / numpy_time)
```

type casting -

```
a = np.array(['1.2', '2.5', '3.6', '4.8'])
```

```
type(a)
```

```
numpy.ndarray
```

```
a.dtype # type of data elements
```

```
dtype('<U3')
```

```
a = a.astype(float)
```

```
a.dtype ('float64')
```

Data exploration

1. • shape
2. • ndim
3. • size